POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern comportamentali

State

Motivazioni

- Si vuole implementare il protocollo TCP
- La sequenza di azioni da compiere è rappresentabile come un automa a stati finiti
- La classe TCPConnection deve gestire l'apertura della connessione, la chiusura e le varie fasi intermedie

Motivazioni

- **Problema:** la gestione di tutte le operazioni da compiere è complessa e fortemente dipendente da uno stato
- La classe TCPConnection può diventare troppo grande
- Molti statement condizionali da gestire

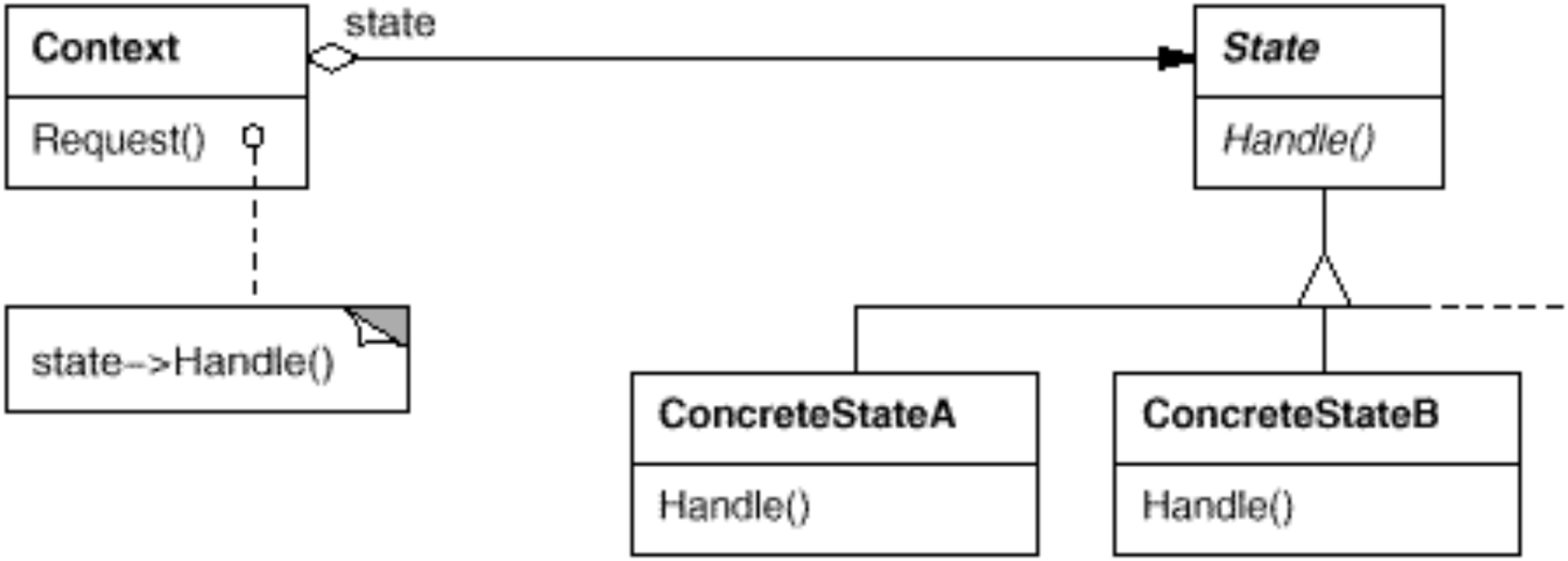
Una soluzione non ideale

```
class TCPConnection {

        int state = 0;

        public void open(){
                if (state == 0){
                    // initilize connection
                    if (success) state = 1;
                    else state = -1;
                }
                else if (state == 1) …
                else if (state == 2) …
                …
        }
        public void close(){ … }
}
```

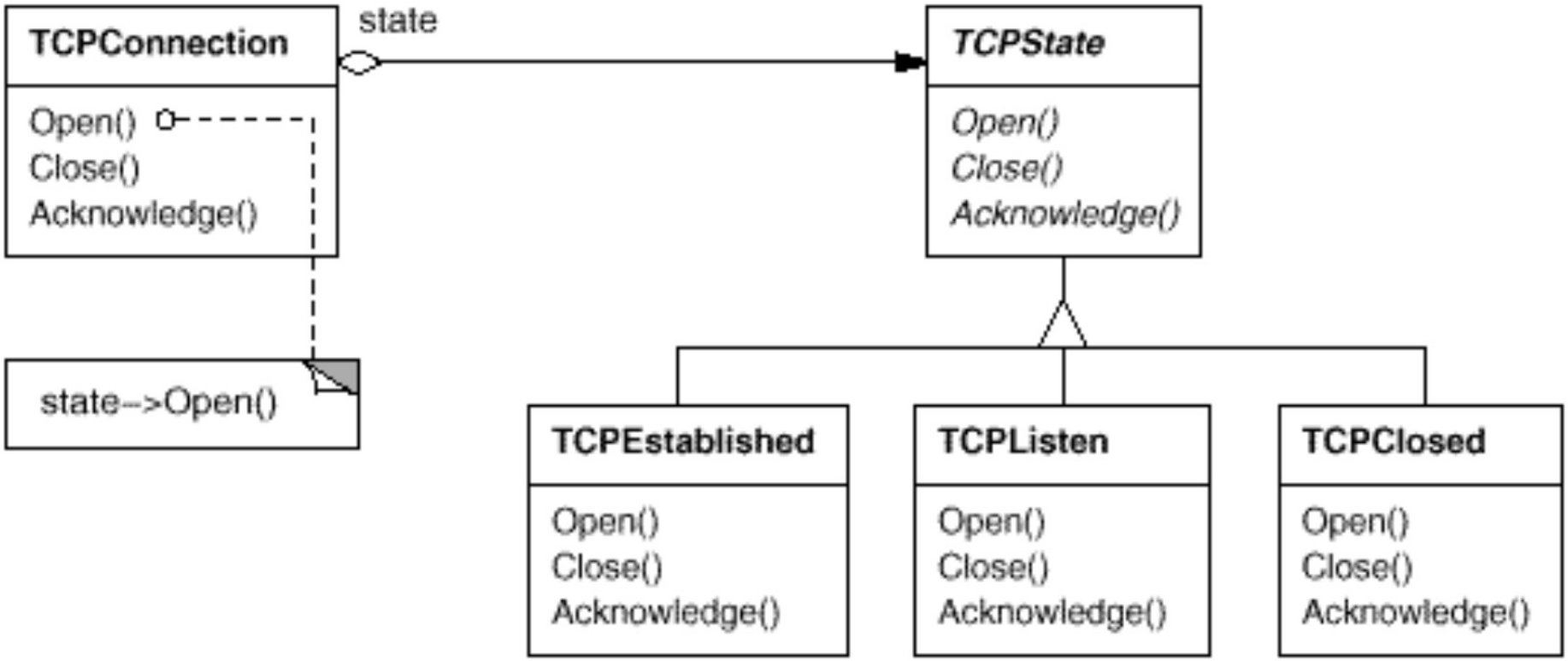
Obbiettivi

- Definire una classe astratta State
- Ciascuna sottoclasse di State:
  - Gestisce le operazioni relative allo stato
  - Effettua la transizione al prossimo stato
- TPCConnection manterrà un riferimento allo stato corrente
- La logica applicativa è suddivisa in diverse classi (più mantenibile)

Struttura


Context
Request()
state
State
Handle()
state->Handle()
ConcreteStateA
Handle()
ConcreteStateB
Handle()

Esempio


TCPConnection
Open()
Close()
Acknowledge()
state
TCPState
Open()
Close()
Acknowledge()
state->Open()
TCPEstablished
Open()
Close()
Acknowledge()
TCPListen
Open()
Close()
Acknowledge()
TCPClosed
Open()
Close()
Acknowledge()

Esempio

```
abstract class TCPState {
        protected TCPConnection connection;
        public TCPState(TCPConnection connection){
                this.connection = connection;
        }
        abstract public void open();
        abstract public void close();
        abstract public void acknowledge();

        public void transition(TCPState state){
                connection.setState(state);
        }

}
```

Esempio

```
public class TCPListen extends TPCState {
      public TCPListen(TCPConnection connection){
            super(connection);
      }
      public void open(){
            // opening logic
            this.transition(new TCPEstablished(connection));
      }
      public void close(){ … }
      public void acknoledge(){ … }

}
```

Esempio

```
class TCPConnection {
        private TPCState state;

        public TPCConnection(){
                state = new TCPListen(this);
        }
        public void setState(TCPState newState){
                state = newState;
        }
        public void open(){
                state.open();
        }
        public void close(){
                state.close();
        }
        public void acknowledge(){
                state.acknowledge();
        }
}
```